

Study Planner API

1. Project Overview

The **Study Planner API** is a backend REST API application developed using **Java, Spring Boot, Spring Data JPA, H2 Database, and Maven**.

The purpose of the project is to help users manage their study tasks through REST APIs.

A user can:

- Create a study task
- View all study tasks
- Filter tasks based on status, priority, or subject
- View a specific task
- Update an existing task
- Mark a task as completed
- Delete a task
- View a summary of study progress

The project follows a **layered architecture**, where different responsibilities are separated into Controller, Service, Repository, Model, and DTO layers.

2. Technologies Used

Technology	Purpose
Java	Programming language
Spring Boot	Backend application framework
Spring Data JPA	Database interaction
H2 Database	Embedded local database
Maven	Project/build dependency management
REST API	Communication through HTTP
Postman	API testing

The project does **not require manually written SQL queries** for the normal CRUD operations. Spring Data JPA handles the database interaction and generates the required SQL automatically.

3. Main Objective

The main objective of the project is to build a backend system that manages study tasks.

For example, a student can create:

```
Task:
Title: Learn Spring Boot
Subject: Java
Priority: HIGH
Status: TODO
Due Date: 2026-10-05
```

The application stores this information in the H2 database.

Later, the user can:

```
View the task
    ↓
Start working on it
    ↓
Change status to IN_PROGRESS
    ↓
Complete the task
    ↓
Change status to COMPLETED
```

4. Project Architecture

The project follows this architecture:

```
Client / Postman
    |
    v
TaskController
    |
    v
TaskService
    |
    v
TaskRepository
```

|
v
H2 Database

Each layer has a specific responsibility.

Controller

The Controller receives HTTP requests from the client.

For example:

```
GET /api/tasks  
POST /api/tasks  
PUT /api/tasks/1  
DELETE /api/tasks/1
```

The controller should not contain the main business logic.

Service

The Service contains the main application logic.

For example:

```
Find a task  
Check whether it exists  
Update task information  
Mark task as completed  
Calculate summary
```

Repository

The Repository communicates with the database using Spring Data JPA.

For example:

```
taskRepository.findById(id);  
taskRepository.save(task);  
taskRepository.delete(task);
```

Spring Data JPA generates the appropriate SQL automatically.

H2 Database

H2 is an embedded database used to store the task information locally.

The database contains the stored task records.

5. Project Files

The important files are:

```
StudyPlannerApplication.java  
Task.java  
TaskController.java  
TaskService.java  
TaskRepository.java  
TaskRequest.java
```

StudyPlannerApplication.java

This is the main class used to start the Spring Boot application.

When the application starts, Spring Boot initializes the application and starts the embedded server.

Task.java

`Task.java` represents the task entity.

It defines the information associated with a study task.

For example:

```
id
title
subject
description
priority
status
dueDate
createdAt
```

The class represents the structure of a task stored in the database.

TaskController.java

This class handles incoming HTTP requests.

For example:

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/1
PUT /api/tasks/1
PATCH /api/tasks/1/complete
DELETE /api/tasks/1
```

TaskService.java

This class contains the main application logic.

For example, when a user requests:

```
PATCH /api/tasks/1/complete
```

the service finds task `1` and changes its status to:

```
COMPLETED
```

TaskRepository.java

This interface is responsible for database operations.

Spring Data JPA provides the implementation automatically.

For example:

```
findById()  
save()  
delete()  
findAll()
```

TaskRequest.java

This class is used for receiving and validating task input.

For example, it can validate that required information such as the title is provided.

6. Important Java Concepts Used

Class

A class is a blueprint.

For example:

```
class Task {  
    String title;  
    String subject;  
}
```

The `Task` class describes what information a task contains.

Object

An object is an actual instance created from a class.

For example:

```
Task task = new Task();
```

Here:

```
Task = class  
task = object
```

Method

A method represents an action.

For example:

```
save()  
delete()  
findById()
```

These methods perform specific operations.

7. REST API

The project exposes REST APIs through HTTP.

The main HTTP methods used are:

HTTP Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
PATCH	Partially update data
DELETE	Delete data

The base URL is:

```
http://localhost:8080/api/tasks
```

8. Create Task API

Endpoint

```
POST /api/tasks
```

This API is used to create a new study task.

Request

```
{
  "title": "Learn Spring Boot",
  "subject": "Java",
  "description": "Understand REST APIs",
  "priority": "HIGH",
  "status": "TODO",
  "dueDate": "2026-10-05"
}
```

Successful Response

The API returns:

```
201 Created
```

Example response:

```
{
  "id": 1,
  "title": "Learn Spring Boot",
  "subject": "Java",
  "description": "Understand REST APIs",
  "priority": "HIGH",
  "status": "TODO",
  "dueDate": "2026-10-05",
}
```

```
"createdAt": "2026-09-28T19:30:00"
}
```

9. POST API Flow

This is one of the most important flows to understand for the project.

Suppose the user sends:

```
POST /api/tasks
```

with a JSON body.

The flow is:

```
Client / Postman
  |
  v
TaskController
  |
  v
TaskRequest
  |
  v
TaskService
  |
  v
TaskRepository
  |
  v
H2 Database
```

Step 1 — Request reaches Controller

The client sends the POST request.

The `TaskController` receives it.

Step 2 — Input Validation

The request data is checked using `TaskRequest` .

For example:

```
Title should not be empty  
Due date should be valid
```

Step 3 — Service Layer

The controller passes the request to `TaskService` .

The service creates the `Task` object and handles the application logic.

Step 4 — Repository

The service calls:

```
taskRepository.save(task);
```

Step 5 — Spring Data JPA

Spring Data JPA handles the database operation.

The developer does not manually write the INSERT SQL query.

Conceptually, JPA performs something similar to:

```
INSERT INTO task (...)  
VALUES (...);
```

Step 6 — H2 Database

The task is stored in the H2 database.

Step 7 — Response

The saved task is returned to the client as JSON.

This complete flow demonstrates how the different layers of the application work together.

10. Get All Tasks API

Endpoint

```
GET /api/tasks
```

This API retrieves all tasks.

Response

```
200 OK
```

The response contains the available tasks.

11. Filtering Tasks

The API also supports filtering.

Filter by Status

```
GET /api/tasks?status=TODO
```

This retrieves tasks whose status is `TODO`.

Filter by Priority

```
GET /api/tasks?priority=HIGH
```

This retrieves high-priority tasks.

Filter by Subject

```
GET /api/tasks?subject=java
```

This retrieves tasks related to Java.

Multiple Filters

Filters can also be combined:

```
GET /api/tasks?status=TODO&priority=HIGH&subject=java
```

This allows the user to find tasks matching multiple conditions.

12. Task Status

The project uses these task statuses:

```
TODO
IN_PROGRESS
COMPLETED
```

Example:

```
TODO
  ↓
IN_PROGRESS
  ↓
COMPLETED
```

This represents the progress of a study task.

13. Task Priority

The supported priorities are:

```
LOW
MEDIUM
HIGH
```

For example:

```
Prepare for interview → HIGH
Read documentation → MEDIUM
Review notes → LOW
```

14. Get Task By ID

Endpoint

```
GET /api/tasks/{id}
```

Example:

```
GET /api/tasks/1
```

This retrieves the task whose ID is `1`.

If the task exists, its details are returned.

If the task does not exist, the application returns a `404 Not Found` response.

Example:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Task with id 1 was not found"
}
```

The project handles this using `TaskNotFoundException`.

15. Update Task API

Endpoint

```
PUT /api/tasks/{id}
```

Example:

```
PUT /api/tasks/1
```

Request:

```
{
  "title": "Complete Spring Boot APIs",
  "subject": "Java",
  "description": "Test all endpoints in Postman",
  "priority": "HIGH",
  "status": "IN_PROGRESS",
  "dueDate": "2026-10-10"
}
```

The application:

```
Find task
  ↓
Update task fields
  ↓
Save task
  ↓
Return updated task
```

The successful response is:

```
200 OK
```

16. Complete Task API

Endpoint

```
PATCH /api/tasks/{id}/complete
```

Example:

```
PATCH /api/tasks/1/complete
```

This endpoint does not require a request body.

Its purpose is to mark the selected task as completed.

For example:

Before:

```
status = IN_PROGRESS
```

After:

```
status = COMPLETED
```

The service finds the task, changes its status, and saves it.

17. Delete Task API

Endpoint

```
DELETE /api/tasks/{id}
```

Example:

```
DELETE /api/tasks/1
```

The application finds the task and deletes it from the database.

Successful response:

204 No Content

There is no JSON response body for a successful delete.

18. Task Summary API

Endpoint

GET /api/tasks/summary

This endpoint provides an overall summary of the study tasks.

Example:

```
{
  "totalTasks": 5,
  "completedTasks": 2,
  "activeTasks": 3,
  "overdueTasks": 1
}
```

The summary provides:

totalTasks

Total number of tasks.

completedTasks

Number of tasks whose status is:

COMPLETED

activeTasks

Tasks that are still active.

overdueTasks

Tasks whose due date has passed and whose status is not `COMPLETED`.

19. Validation

The application also validates incoming task data.

For example:

```
{
  "title": "",
  "subject": "Java",
  "priority": "HIGH",
  "dueDate": "2020-01-01"
}
```

This request is invalid because:

```
Title is empty
Due date is in the past
```

The API returns:

```
400 Bad Request
```

The validation is handled through `TaskRequest`.

20. Database Interaction

One important point to understand is that the project uses:

```
Spring Data JPA
```

instead of manually writing SQL for normal CRUD operations.

For example:

```
taskRepository.findById(id);
```

conceptually results in a query similar to:

```
SELECT *  
FROM task  
WHERE id = ?;
```

Similarly:

```
taskRepository.save(task);
```

can result in an:

```
INSERT
```

or:

```
UPDATE
```

depending on whether the entity is new or already exists.

And:

```
taskRepository.delete(task);
```

results in a delete operation.

21. Why Layered Architecture?

The project separates responsibilities.

Instead of putting everything inside the Controller:

```
Controller
  ↓
Everything
```

the project uses:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

This makes the application easier to understand and maintain.

Controller

Handles HTTP requests.

Service

Handles application logic.

Repository

Handles database operations.

Database

Stores the actual data.

22. Complete Example

Suppose a student wants to create a task:

```
Learn Spring Boot
```

They send:

```
POST /api/tasks
```

with:

```
{
  "title": "Learn Spring Boot",
  "subject": "Java",
  "description": "Understand REST APIs",
  "priority": "HIGH",
  "status": "TODO",
  "dueDate": "2026-10-05"
}
```

The application processes the request:

```
Postman
  ↓
TaskController
  ↓
TaskRequest validation
  ↓
TaskService
  ↓
TaskRepository
  ↓
Spring Data JPA
  ↓
H2 Database
```

The task is stored.

Later, the user can retrieve it:

```
GET /api/tasks/1
```

Start working on it:

```
PUT /api/tasks/1
```

or mark it completed:

```
PATCH /api/tasks/1/complete
```

Finally, if the task is no longer required:

```
DELETE /api/tasks/1
```

23. How to Explain the Project in an Interview

A simple explanation is:

"My project is a Study Planner REST API developed using Java and Spring Boot. It allows users to create, view, update, delete, filter, and complete study tasks. I used a layered architecture with Controller, Service, Repository, Model, and DTO layers. Spring Data JPA is used for database interaction, and H2 is used as the embedded database. The controller handles HTTP requests, the service contains the application logic, and the repository communicates with the database."

24. How to Explain the Architecture

If the interviewer asks:

"Explain the architecture of your project."

You can say:

"The project follows a layered architecture. The Controller receives HTTP requests from the client. It passes the request to the Service layer, where the main business logic is handled. The Service communicates with the Repository layer. The Repository uses Spring Data JPA to interact with the H2 database. This separation keeps the responsibilities of each layer clear."

25. How to Explain the POST Request

If the interviewer asks:

"What happens when you create a task?"

You can explain:

"When the client sends a POST request to `/api/tasks`, the request first reaches the TaskController. The request data is validated using TaskRequest. Then the controller passes the data to TaskService. The service creates the Task object and passes it to TaskRepository. Spring Data JPA then handles the database operation and stores the task in the H2 database. Finally, the saved task is returned as a JSON response."

26. How to Explain Spring Data JPA

If the interviewer asks:

"Did you write SQL queries?"

Answer:

"For the normal CRUD operations, I did not manually write SQL queries. I used Spring Data JPA repository methods such as `findById()`, `save()`, `delete()`, and `findAll()`. Spring Data JPA generates the required SQL queries internally."

27. How to Explain H2

If asked:

"Why did you use H2?"

You can say:

"I used H2 as an embedded database for local development and testing. It is easy to configure and allows me to test the REST APIs without setting up an external database server."

28. What You Should Learn First

If you are learning this project from scratch, follow this order:

Step 1 — Java Basics

Learn:

```
Class
Object
Constructor
Method
if/else
Loops
ArrayList
```

Step 2 — HTTP Basics

Understand:

```
GET
POST
PUT
PATCH
DELETE
```

Step 3 — Understand Task.java

Understand:

```
What is a class?
What are fields?
What is an object?
What is a constructor?
```

Step 4 — Understand TaskController

Learn how:

```
GET
POST
PUT
PATCH
DELETE
```

are mapped to Java methods.

Step 5 — Understand TaskService

Follow the request from the controller into the service.

Understand what happens to the data.

Step 6 — Understand Repository

Learn:

```
findById()  
findAll()  
save()  
delete()
```

Step 7 — Understand H2 + JPA

Understand how Java objects become database records and how Spring Data JPA handles database operations.

Step 8 — Test Everything Using Postman

Practice:

```
POST  
GET  
GET by ID  
PUT  
PATCH  
DELETE  
SUMMARY
```

29. Recommended Practice Order

Start with only these three APIs:

1. Create

```
POST /api/tasks
```

2. View

```
GET /api/tasks
```

3. Complete

```
PATCH /api/tasks/1/complete
```

Once these are understood, move to:

```
GET by ID  
PUT  
DELETE  
Filters  
Summary  
Validation  
Exception handling
```

30. Final Project Understanding

The most important thing to understand is not memorizing every line of code.

Understand this flow:

```
HTTP Request  
↓  
Controller  
↓  
Service  
↓  
Repository  
↓  
H2 Database  
↓  
Repository
```

```
↓  
Service  
↓  
Controller  
↓  
JSON Response
```

Once this flow is clear, the entire **Study Planner API** becomes much easier to understand.

The core idea of the project is:

```
Create a task  
↓  
Store it  
↓  
Retrieve it  
↓  
Update it  
↓  
Complete it  
↓  
Delete it
```

This demonstrates the fundamental concepts of a **Java Spring Boot REST API with Spring Data JPA and H2**.